# KANBAN STUDIO PRO ■

A-Z Comprehensive Architectural Documentation, File Reference & Deployment Guide

## 1. Executive Summary & Application Overview

**Kanban Studio Pro** is a full-stack, enterprise-grade project management web application. It features dynamic drag-and-drop task boards, multi-project workspace isolation, granular Role-Based Access Control (RBAC), cryptographic session token security, real-time WebSocket synchronization across devices, and an embedded natural-language AI Kanban Assistant with OpenRouter model failover.

## 2. Complete Technology Stack

| Layer | Technologies & Version | Purpose / Key Features |
|---|---|---|
| Frontend | Next.js 16 (Turbopack), React 19, TypeScript, TailwindCSS v4 | High-performance App Router SPA with server/client components and modern glassmorphism styling. |
| Drag & Drop | @dnd-kit/core, @dnd-kit/sortable | Accessible, touch-friendly task reordering with MouseSensor, PointerSensor & TouchSensor (150ms delay). |
| Backend API | FastAPI, Python 3.13, Uvicorn ASGI Server | Asynchronous REST API endpoints, WebSocket connection manager, rate limiting, and CORS routing. |
| Database | SQLite3 (WAL Mode), Python sqlite3 module | Relational persistence for users, sessions, projects, columns, cards, member roles, activity logs, and notifications. |
| Security | Cryptographic Session Tokens, PBKDF2 Hashing, RBAC Guard | secrets.token_hex(32) session tokens, logout revocation, IDOR prevention, and role hierarchy enforcement. |
| AI Assistant | OpenRouter API (GPT-4o-mini) + Model Failover Stack | Parses natural language prompts with failover (GPT-4o-mini -> Llama 3.3 70B -> Auto -> Smart Local NLP). |
| DevOps & Cloud | Docker, Render.com, Vercel, GitHub Actions | Containerized Python backend on Render + static Edge CDN frontend hosting on Vercel. |
| Testing | Pytest (38), Vitest (44), Playwright E2E (14) | Comprehensive 96-test automated suite covering backend API, security audit, frontend UI, and E2E browser flows. |

## 3. Complete Directory & File Structure (A-Z Guide)

Below is a breakdown of every single file in the repository and its exact technical responsibility:

- **Dockerfile (Root)**: Multi-stage Python 3.13 container definition for building and running the FastAPI backend on cloud hosts (Render/Railway). Installs system dependencies, copies requirements, and launches uvicorn on port 8000.
- **render.yaml (Root)**: Render Blueprint deployment configuration file. Configures automatic container builds from root Dockerfile and passes CORS_ORIGINS and PORT environment variables.
- **README.md (Root)**: Project documentation detailing live demo links, architecture overview, screenshot previews, and setup instructions.
- **pm/backend/main.py**: FastAPI application entry point. Defines REST endpoints (/api/auth/register, /api/auth/login, /api/projects, /api/board, /api/cards, /api/ai/chat) and authenticated WebSocket route (/ws/projects/{id}).
- **pm/backend/database.py**: Core SQLite database layer. Contains schema definitions, sessions table, PBKDF2 password hashing, RBAC permission checks (Owner, Admin, Member, Viewer, None), CRUD operations, and default board seeding.
- **pm/backend/ai.py**: AI Assistant logic. Integrates OpenRouter API with model failover stack (gpt-4o-mini -> llama-3.3-70b -> auto -> smart local NLP) to execute structured board mutations.
- **pm/backend/websocket_manager.py**: ConnectionManager class managing real-time WebSocket client connections and broadcasting board updates to connected project members.
- **pm/backend/config.py**: Environment configuration loader with override=True for reading OPENROUTER_API_KEY, database paths, CORS origin lists, and API keys.
- **pm/backend/requirements.txt**: Backend Python dependencies list (fastapi, uvicorn, pydantic, pytest, python-dotenv, httpx).
- **pm/backend/test_security_audit.py & test_part28_adversarial_security.py**: Pytest security regression test suite validating session token security, token revocation on logout, IDOR prevention, RBAC role restrictions, unauthenticated WebSocket rejection, and AI prompt injection resistance.
- **pm/docs/FINAL_RELEASE_CERTIFICATION.md**: Final Release Certification report confirming PRODUCTION READY status backed by 82+ automated test passes.
- **pm/frontend/src/app/page.tsx & layout.tsx**: Next.js App Router root layout and primary page shell rendering the main Kanban interface.
- **pm/frontend/src/app/globals.css**: Global TailwindCSS v4 stylesheet containing modern theme CSS variables, glassmorphism card utilities, glowing animations, and vertical mobile column layout rules.
- **pm/frontend/src/components/KanbanBoard.tsx**: Core frontend board orchestrator. Manages user auth state, project switcher, undo/redo history, WebSocket real-time sync, @dnd-kit sensors, and column grid.
- **pm/frontend/src/components/AIAssistantWidget.tsx**: Floating AI Assistant chat drawer. Connects to /api/ai/chat via getApiUrl and automatically applies server-returned board updates.
- **pm/frontend/src/components/LoginForm.tsx**: Sign-in and Create Account tabbed modal featuring auto-login, error alerts, case normalization, and fallback client registration.
- **pm/frontend/src/lib/api.ts**: API client module exporting getApiUrl, fetchBoard, saveBoard, registerApi, and project management network handlers with Bearer token authentication.

- **pm/frontend/src/lib/useUndoRedo.ts**: Custom React hook providing multi-level Undo (Ctrl+Z) and Redo (Ctrl+Y) state history management.
- **pm/frontend/tests/kanban.spec.ts**: Playwright E2E browser test suite (14 tests) running automated user interactions across desktop and mobile viewports.

## 4. Core Site Logic & Key Functionalities

**Authentication & Session Management**: Users sign in or register new accounts. Username strings are automatically sanitized and lower-cased. Passwords are salted and hashed using PBKDF2-HMAC-SHA256 (100,000 iterations). Active sessions issue secrets.token_hex(32) tokens, and logout revokes tokens in SQLite.

**Multi-Project Isolation & IDOR Protection**: Users create, rename, switch between, and delete independent Kanban projects. Every backend API endpoint validates project permissions before allowing access or mutation.

**Drag & Drop Engine (Desktop + Mobile)**: Powered by @dnd-kit. Features PointerSensor, MouseSensor, and TouchSensor (with a 150ms delay and 5px tolerance). On mobile screens, columns stack vertically to eliminate horizontal scroll issues.

**Security & RBAC Enforcement**: Backend endpoints enforce role hierarchy checks (Owner > Admin > Member > Viewer > None). Card mutations verify user membership before executing SQLite updates.

**Real-Time WebSockets Sync**: When multiple users collaborate on the same project, board updates (card moves, edits, deletions) are instantly broadcast via WebSockets (/ws/projects/{id}) with token authentication.

**AI Kanban Assistant & Model Failover**: Embedded chat drawer enables users to manage their board with natural language. Powered by OpenRouter API with multi-model failover stack (gpt-4o-mini -> llama-3.3-70b-instruct -> auto -> local NLP).

**Activity Log & Notification System**: Every project action (adding cards, editing titles, changing priority) is logged in SQLite and presented in interactive modal dialogs.

## 5. Complete Testing Methodology (96 Automated Tests)

The application is validated by an automated test suite comprising **96 total tests** across 3 distinct test runners:

- **Pytest (38 Tests)**: Validates backend REST routes, SQLite database schemas, PBKDF2 password hashing, cryptographic session tokens, RBAC permission checks, IDOR isolation, AI prompt injection resistance, and Part 28 adversarial security scenarios.
- **Vitest (44 Tests)**: Unit tests frontend helper utilities (filterAndSortBoard, moveCard, useUndoRedo) and React UI components (LoginForm, TaskFilterToolbar, ProjectSwitcher).
- **Playwright E2E (14 Tests)**: Automated end-to-end browser tests in Headless Chromium. Verifies full user sign-in, task creation, dragging cards across columns, mobile viewport responsiveness, and multi-user login workflows.

# 6. DevOps Architecture: Docker, Render & Vercel

**Why Docker?**
Docker packages the Python runtime, FastAPI dependencies, Uvicorn server, and SQLite database into a standardized, lightweight Linux container. This eliminates 'it works on my machine' issues and ensures identical execution across local dev and cloud servers.

**Render Cloud Backend Hosting**:
Render reads the root render.yaml and builds the Docker container. It exposes port 8000 and serves CORS-enabled API requests. Note: Render free instances spin down after 15 minutes of inactivity; the initial request takes ~30 seconds for cold-start initialization.

**Vercel Frontend Hosting & Environment Variables**:
Vercel hosts the Next.js static production bundle on an Edge CDN. The environment variable NEXT_PUBLIC_API_URL points to the Render backend URL (https://drag-n-drop-28p3.onrender.com). Because Next.js inlines NEXT_PUBLIC_ variables at build time, updating this variable triggers a fresh Vercel rebuild to embed the live endpoint into the web app.